

EduAgent: A Personalized Agentic AI Tutor with Fine-Tuned Difficulty Classification, Semantic Two-Pass RAG, Learner Memory, Voice Interaction, and Adaptive Teaching Modes

Research Paper-Style Technical Report

Department of Artificial Intelligence
Third Year B.Tech GenAI Project

May 2026

Abstract

EduAgent is a personalized AI/ML tutoring system that moves beyond one-shot chatbot behavior by combining a fine-tuned DistilBERT difficulty classifier, semantic topic detection, two-pass personalized retrieval, persistent learner memory, LLM-based tutoring and evaluation agents, and a Gradio interface with voice input/output. The updated repository uses a 2,400-row Gemini-generated instructional dataset with topic, subtopic, difficulty, angle, reasoning, and confusion metadata. A DistilBERT classifier trained on this dataset reaches 97.92% test accuracy and 97.91% macro-F1 on a balanced 240-example test split. The current saved RAG checkpoint evaluates 18 questions and reports Hit Rate@3 = 83.3%, MRR = 0.8333, Topic Precision = 83.3%, Level Precision = 100%, average semantic similarity = 0.7153, Faithfulness = 5.00/5, Answer Relevancy = 5.00/5, Context Utilization = 4.44/5, and Level Fit = 4.94/5. A separate weak-area retrieval evaluation reports 100% keyword relevance over five simulated learner weakness profiles. This report documents the architecture, dataset construction, prompts, classifier results, RAG evaluation, memory update equations, testing evidence, limitations, and future research directions.

Contents

1	Introduction	3
2	Problem Statement	3
3	Research Gap	4
4	Objectives	4
5	Why This is an Agentic AI Project	5
5.1	Agentic Characteristics	5
5.2	Comparison With a Plain LLM Tutor	6
6	System Architecture	6
6.1	High-Level Architecture Overview	6
6.2	Agent Communication Diagram	8
7	Novelty 1 — Shared Semantic Embedder	8
7.1	Design Decision	8
7.2	The Shared Embedder Module	8

8 Novelty 2 — Semantic Topic Detection	9
8.1 Previous Approach (TF-IDF)	9
8.2 New Approach (Dense Embeddings)	9
8.3 Alias Table (Fast Path)	10
8.4 Results	10
9 Novelty 3 — Two-Pass Personalized RAG	11
9.1 What RAG Means in EduAgent	11
9.2 Limitation of Single-Pass RAG	11
9.3 Two-Pass Architecture	11
9.4 Prompt Injection	12
9.5 Index Caching	12
9.6 Configurable Retrieval Sizes	13
10 Novelty 4 — Four Adaptive Teaching Modes	13
10.1 Overview	13
10.2 Mode Selection Logic	14
10.3 Mode-Specific Prompt Instructions	14
11 Novelty 5 — Mastery Model with Diminishing Returns	15
11.1 Problem with Linear Mastery Scoring	15
11.2 Diminishing Returns Formula	15
12 Novelty 6 — Used-Explanation Tracking	16
12.1 Problem	16
12.2 Implementation	16
13 Dataset	17
13.1 Overview	17
13.2 Coverage	17
13.3 Dataset Columns	17
13.4 Question Generation and Quality Control	17
14 Difficulty Classifier	18
14.1 Model	18
14.2 Classifier Fix: AutoTokenizer	18
14.3 Fallback Classifier	18
14.4 Classification Flow	19
15 Tutor Agent	20
15.1 Inputs	20
15.2 Tutor Agent Prompt	20
15.3 LLM Configuration	21
16 Evaluator Agent	21
16.1 Follow-Up Question Prompt	21
16.2 Learner Reply Evaluation Prompt	21
16.3 Pydantic Validation	22

17 Memory Agent	22
17.1 Learner Profile Schema	22
17.2 Memory Hint Generation	23
17.3 Profile Persistence	23
18 Authentication and Security	23
19 Application Layer	23
19.1 Gradio 6.0 UI	23
19.2 Dashboard Tabs	24
20 Personalization Evaluation	24
20.1 Methodology	24
20.2 Results	25
20.3 Answer Excerpt Evidence	25
21 Unit Testing	25
22 Module Summary	27
23 End-to-End Workflow	28
24 Limitations	28
25 Future Work	29
26 Conclusion	29

1 Introduction

LLM-based educational assistants often answer the current question but do not reliably remember the learner, detect the learner’s difficulty level, retrieve level-appropriate examples, check understanding, or adapt the next explanation. EduAgent addresses this limitation by wrapping LLM generation inside an agentic tutoring loop:

Ask → Classify → Retrieve → Teach → Evaluate → Remember → Adapt.

The updated EduAgent repository implements this loop as a working application rather than a static notebook. It includes authentication, SQLite-backed learner profiles, a local DistilBERT classifier, semantic retrieval using `all-MiniLM-L6-v2`, LLM-based Tutor and Evaluator agents, an adaptive Memory Agent, voice input/output modules, dashboard visualizations, and evaluation scripts.

1.1 Main Research Contributions

1. **Dataset upgrade:** Replaced a compact 900-row baseline dataset with a 2,400-row metadata-rich AI/ML tutoring dataset: 25 topics × 4 subtopics × 3 difficulty levels × 8 examples.
2. **Fine-tuned difficulty classifier:** Trained `distilbert-base-uncased` for 3-class difficulty prediction, achieving 97.92% test accuracy and 97.91% macro-F1.
3. **Two-pass personalized RAG:** Pass 1 retrieves examples similar to the learner query; Pass 2 retrieves weak-area examples from the learner profile.
4. **Adaptive teaching modes:** Four modes - default, remedial, clarification, and advance - are selected from evaluator output, mastery scores, weak concepts, and prior explanation styles.
5. **Closed feedback loop:** Follow-up evaluation updates weak areas, mastery, recommendations, and the prompt strategy used in later turns.
6. **Operational application:** The project includes authentication, SQLite persistence, Gradio UI, learner dashboard charts, voice input, voice output, retry-aware LLM client, and unit tests.

2 System Architecture

2.1 End-to-End Pipeline

Figure 1 summarizes the updated repository workflow. Blue boxes are deterministic/system components, green boxes are retrieval and memory components, orange boxes are LLM agents, and cylinders represent persistent or reusable data stores.

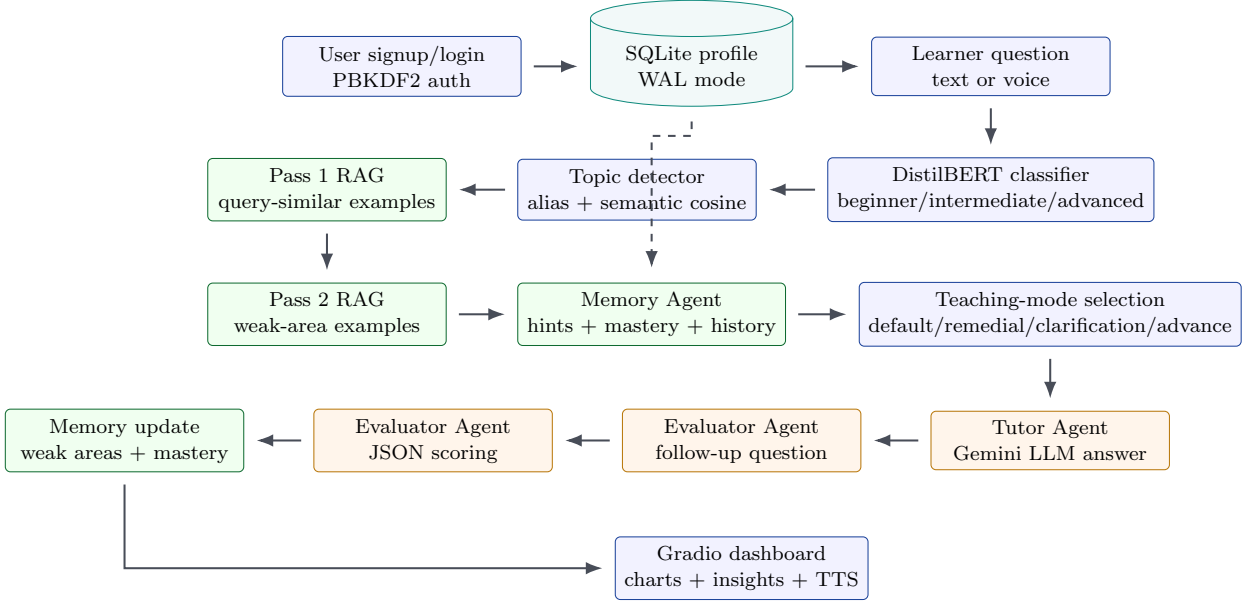


Figure 1: EduAgent end-to-end architecture. The dashed arrow represents cross-turn profile reads.

2.2 Repository Module Map

Module	Role in the system
<code>config/settings.py</code>	Central configuration: Gemini key/model, dataset path, classifier path, DB file, RAG top-N values, evaluation model, retry settings.
<code>ml/classifier.py</code>	Loads local DistilBERT model from <code>models/distilbert_eduagent_v2</code> , falls back to Hugging Face repo, then heuristic classification. Adds calibration for beginner, comparison, and advanced-intent phrasing.
<code>ml/embedder.py</code>	Loads shared <code>all-MiniLM-L6-v2</code> embedder from local cache and exposes a singleton to avoid loading the sentence-transformer twice.
<code>ml/topic_detector.py</code>	Detects canonical AI/ML topic using alias matching first, then semantic topic vectors or TF-IDF fallback.
<code>ml/retriever.py</code>	Implements level/topic filtering, semantic or TF-IDF retrieval, keyword overlap scoring, complexity penalty, and weak-area retrieval.
<code>agents/tutor_agent.py</code>	Orchestrates classification, topic detection, Pass 1 RAG, Pass 2 RAG, memory hints, teaching-mode inference, prompt assembly, and answer generation.
<code>agents/evaluator_agent.py</code>	Generates a conceptual follow-up question and evaluates learner replies into a validated schema: good , partial , or poor .
<code>agents/memory_agent.py</code>	Maintains profile schema, updates topic counts, mastery, weak areas, used explanation tags, recommendations, and prompt hints.
<code>agents/llm_client.py</code>	Wraps Gemini calls, OpenAI-style messages, retry logic, timeout, quota detection, and fallback behavior.
<code>auth/password_utils.py</code>	PBKDF2-SHA256 password hashing with 16-byte salts and <code>hmac.compare_digest</code> .
<code>db/sqlite_store.py</code>	SQLite initialization with WAL mode, synchronous=NORMAL, users table, profiles table, and schema migration checks.

Module	Role in the system
db/profile_repository.py	User CRUD and profile JSON serialization/deserialization with safe JSON defaults.
app/main.py	Gradio callback orchestration: login, signup, question asking, follow-up evaluation, profile save, chart generation, retrieved-example display, voice callbacks.
app/ui.py	Full Gradio layout and CSS: auth hero, chat panel, dashboard cards, charts, retrieved examples, system insights, voice controls.
voice/	Whisper speech-to-text wrapper, local TTS wrapper using Piper if available or pyttsx3 fallback, and UI utilities.
eval/	and Retrieval/generation evaluation, LLM-as-judge scoring, check-pointing, pass-2 weak-area evaluation.
rag_evaluation.py	
tests/	
	Unit tests for authentication, profile normalization, and memory/mastery behavior.

Table 1: Updated repository module map.

3 Dataset Construction and Quality

3.1 Dataset Used by the Updated Application

The active dataset path in `config/settings.py` is:

```
DATASET_FILE = str(BASE_DIR / "datasets" / "data_easy" / "eduagent_dataset_easy_v2.csv")
```

The submitted dataset has 2,400 rows and 19 columns. Its generation target is exactly balanced:

$$25 \text{ topics} \times 4 \text{ subtopics} \times 3 \text{ levels} \times 8 \text{ examples} = 2400.$$

Property	Value in updated repo
Rows	2,400
Columns	19
Difficulty levels	beginner, intermediate, advanced
Rows per level	800 each
Topics	25
Rows per topic	96 each
Subtopics	99 unique subtopic strings
Examples per topic-subtopic-level cell	8
Duplicate questions	0
Duplicate answers	0
Average question length	22.60 words
Average answer length	119.16 words
Generator provider/model	Gemini / <code>gemini-2.5-flash</code>
Prompt version stored in data	<code>eduagent_easy_batch_v1</code>

Table 2: Dataset statistics computed from `datasets/data_easy/eduagent_dataset_easy_v2.csv`.

3.2 Metadata Schema

The dataset is not only a question-answer set; it is a pedagogy-aware instruction dataset. Its columns include:

Column group	Examples
Identity and taxonomy	id, topic, subtopic, level, question_angle
Instruction content	question, answer
Length diagnostics	question_word_count, answer_word_count
Pedagogical annotations	key_concepts, expected_reasoning, difficulty_rationale, why_this_fits_level, concepts_covered, possible_learner_confusion
Generation metadata	generator_provider, generator_model, prompt_version, timestamp

These extra fields are useful for retrieval, reportability, curriculum-aware filtering, and future instruction-tuning experiments.

3.3 Comparison with the Original 900-Row Dataset

The updated dataset is substantially richer than the original 900-row dataset. The detailed analysis document reports the following structural comparison.

Metric	Dataset 1: 900 rows	Dataset 2: 2400 rows
Rows	900	2,400
Columns	5	19
Question avg. words	22.0	22.6
Answer avg. words	62.13	119.16
Question duplicate %	0.0	0.0
Answer duplicate %	0.0	0.0
Question vocabulary size	2,025	3,706
Answer vocabulary size	4,474	8,273
Metadata richness	Low	Very high

Table 3: Dataset comparison from the detailed dataset analysis, verified against the repository dataset where possible.

3.4 Batch Dataset Generation Pipeline

The updated generator is `datasets/generate_dataset_v2.py`. It uses Gemini structured batch output but keeps deterministic local filters in the hot loop. The script design is documented in its header as follows: generate multiple candidate Q/A rows per topic-subtopic-level cell, apply local filters, save continuously to JSONL and CSV, and avoid model self-validation inside the hot loop to reduce wasted calls.

3.4.1 Generation Taxonomy

Level	Question angles in the updated generator
Beginner	definition with simple example; real-world analogy; why it matters; common misconception; basic comparison; high-level working; first learning step; simple consequence
Intermediate	implementation detail; trade-off analysis; debugging scenario; evaluation decision; dataset impact; workflow choice; comparison between methods; practical failure mode
Advanced	edge case analysis; scalability concern; theoretical intuition; research limitation; robustness concern; system design integration; probabilistic interpretation; complexity analysis

Table 4: Level-specific question-angle bank used by the generator.

3.4.2 Local Quality Filters

The script enforces level leakage rejection, length bounds, duplicate/near-duplicate checks, and generic-pattern rejection for intermediate/advanced rows.

Level	Question word range	Answer word range
Beginner	7–45	50–130
Intermediate	9–60	75–170
Advanced	11–75	95–230

Table 5: Deterministic local length filters from the updated dataset generator.

Near-duplicates are rejected when `SequenceMatcher` ratio ≥ 0.90 or Jaccard token similarity ≥ 0.82 within the same topic-subtopic-level cell.

3.5 Dataset Generation Prompt

The full prompt generated by `build_batch_prompt()` is parameterized by `topic`, `subtopic`, `level`, `candidate_count`, and existing questions. The core template is:

```
You are generating high-quality rows for an AI/ML tutoring dataset called EduAgent.

Generate {candidate_count} diverse candidate rows.
Only generate rows for this exact cell:
Topic: {topic}
Subtopic: {subtopic}
Difficulty level: {level}

Existing questions to avoid:
{existing_block}

Use these angles across the rows where possible:
{angles}

Hard quality rules:
- Each question must be specific, useful, and end with a question mark.
- Question length must be {q_min} to {q_max} words.
- Answer length must be {a_min} to {a_max} words.
- Do not mention the difficulty label in the question or answer.
- Do not write "For a beginner", "For an intermediate", or "For an advanced".
- Do not use markdown, bullets, numbered lists, tables, or code blocks.
- Avoid generic questions like "What is {subtopic}?" unless the beginner row adds context and learning value.
- Each answer must be factually correct, self-contained, and pedagogically useful.
- Questions must be meaningfully different from each other and from the existing questions.
```

Level expectations:
 Beginner: simple explanation, intuitive example, no equations, minimal jargon.
 Intermediate: practical reasoning, comparison, debugging, implementation, or evaluation.
 Advanced: edge cases, assumptions, scalability, theory, robustness, or system design reasoning.

Return valid JSON only in this exact shape:

```
{
  "rows": [
    {
      "question": "...",
      "answer": "...",
      "question_angle": "...",
      "key_concepts": ["...", "..."],
      "expected_reasoning": "...",
      "possible_learner_confusion": "...",
      "difficulty_rationale": "...",
      "why_this_fits_level": "..."
    }
  ]
}
```

4 Difficulty Classification

4.1 Classifier Design

The updated repository uses a local fine-tuned `distilbert-base-uncased` sequence classifier stored at:

```
models/distilbert_eduagent_v2
```

The model predicts three labels: `beginner`, `intermediate`, and `advanced`. `ml/classifier.py` first attempts to load the local model. If local weights are unavailable, it attempts a Hugging Face Hub fallback via `CLASSIFIER_HF_REPO`. If both fail, the system falls back to deterministic heuristics. The classifier removes `token_type_ids` before inference because DistilBERT does not use segment embeddings.

4.2 Training Configuration

The saved `training_config.json` in the repository records:

Parameter	Value
Base model	<code>distilbert-base-uncased</code>
Epochs	3
Batch size	8
Max length	128
Learning rate	2×10^{-5}
Weight decay	0.01
Warmup ratio	0.06
Early stopping patience	2
Random seed	42
Train/validation/test split	80/10/10 stratified by level

Table 6: Training configuration saved by the updated repository.

The train/validation/test files saved in the model folder contain 1,920, 240, and 240 examples respectively.

4.3 Classifier Results

The repository classification report gives the following test-set performance:

Class	Precision	Recall	F1	Support
Beginner	0.9875	0.9875	0.9875	80
Intermediate	0.9870	0.9500	0.9682	80
Advanced	0.9639	1.0000	0.9816	80
Accuracy			0.9792	240
Macro average	0.9795	0.9792	0.9791	240
Weighted average	0.9795	0.9792	0.9791	240

Table 7: Fine-tuned DistilBERT test performance.

The confusion matrix is:

	Pred. beginner	Pred. intermediate	Pred. advanced
True beginner	79	1	0
True intermediate	1	76	3
True advanced	0	0	80

Advanced examples are never misclassified as easier levels in the saved test split. Intermediate has the highest confusion, with three intermediate examples predicted as advanced.

4.4 Calibration Layer

The repository does not blindly trust the neural classifier in every case. It adds a small post-prediction calibration layer:

- Short definition-style questions can be forced to beginner.
- Ordinary comparison questions can be treated as intermediate.
- Comparison questions with convergence, derivation, proof, formal, loss landscape, or complexity markers can be upgraded to advanced.
- Explicit advanced markers such as `derive`, `prove`, `theorem`, `convergence`, `architecture`, `attention mechanism`, `backpropagation`, and `gradient flow` can override raw predictions.

This makes the deployed classifier more robust to short and ambiguous real learner queries than a pure argmax classifier.

5 Semantic Topic Detection

The topic detector uses a two-tier design:

1. **Alias tier:** space-padded exact substring matching against a manually curated alias table. This handles terms such as `cnn`, `rag`, `llm`, `tfidf`, `transformer architecture`, and `bias mitigation`.
2. **Semantic tier:** for each topic in the current level slice, the system builds a rich topic document of the form `topic name: all questions for that topic`. It encodes topic documents with the shared `all-MiniLM-L6-v2` embedder and selects the highest cosine-similarity topic when the score is convincing.

If the sentence-transformer cannot load locally, the detector falls back to a TF-IDF topic index. If all similarity scores are too low, the default topic is selected from general topics such as machine learning basics, neural networks, or classification.

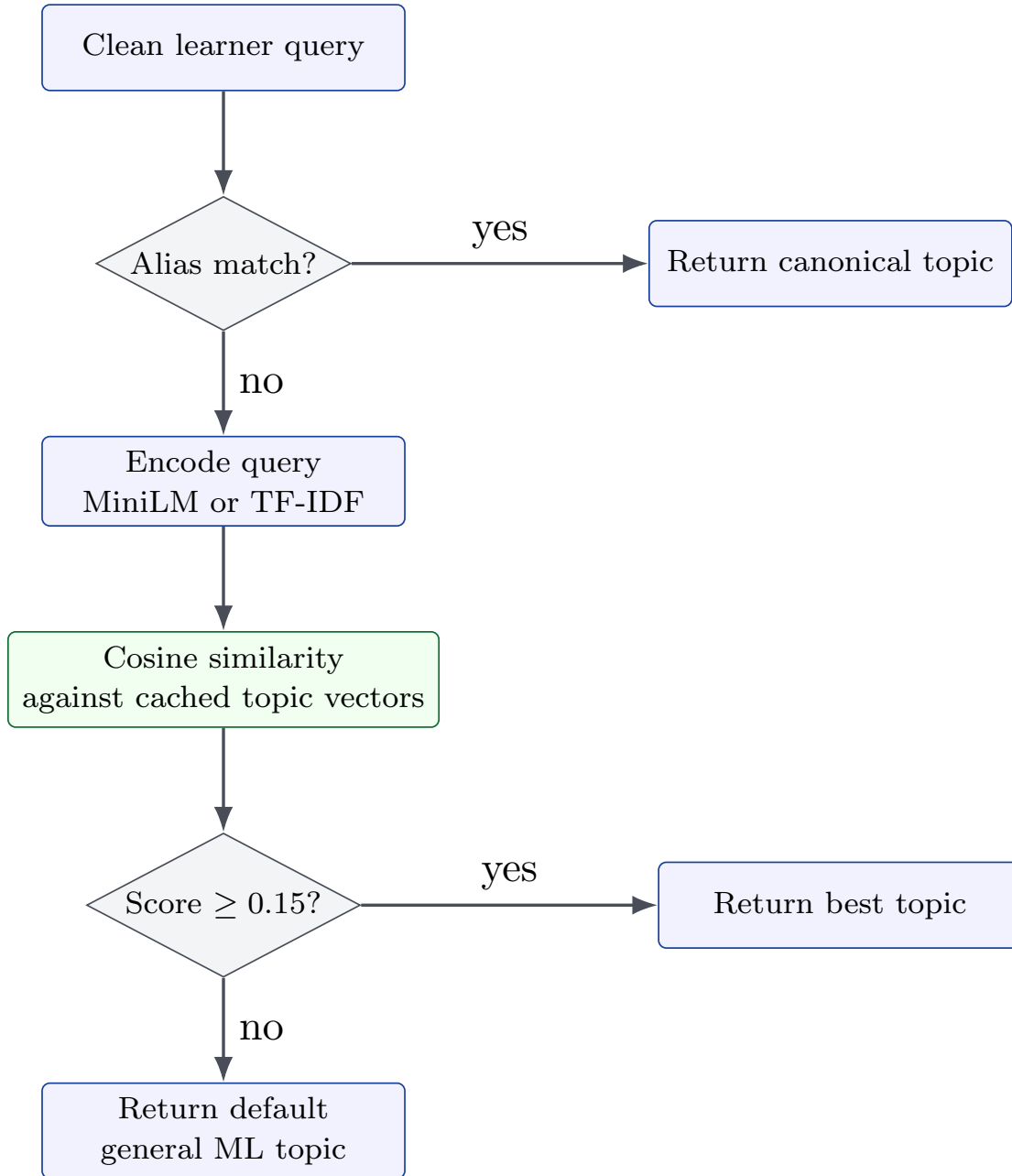


Figure 2: Updated two-tier topic detection: alias matching followed by semantic or TF-IDF fallback.

6 Two-Pass Personalized RAG

6.1 Pass 1: Query-Similar Educational Retrieval

`ml/retriever.py` loads the active dataset once and caches an index for each `(level, topic)` slice. For semantic retrieval, it encodes questions using `all-MiniLM-L6-v2`. If sentence-transformers are unavailable, the app falls back to TF-IDF with unigrams and bigrams.

The final score combines semantic similarity, keyword overlap, and complexity penalty:

$$\text{score} = 0.75 \cdot \text{semantic_cosine} + 0.25 \cdot \text{keyword_overlap} - \text{complexity_penalty}.$$

The complexity penalty is designed to prevent overly difficult examples from being selected for simpler queries. It adds 0.15 when the candidate question is long and another 0.15 for hard terms such as `derive`, `proof`, `convergence`, `subgradient`, or `vanishing gradient`.

6.2 Pass 2: Weak-Area Retrieval

Pass 2 is triggered when the learner profile contains weak concepts for the detected topic. For each weak concept, the query is:

```
query = f"{concept} {topic}".strip()
```

The retriever pools results across weak concepts, removes duplicates and rows already returned in Pass 1, and returns up to `RAG_WEAK_TOP_N=2` examples.

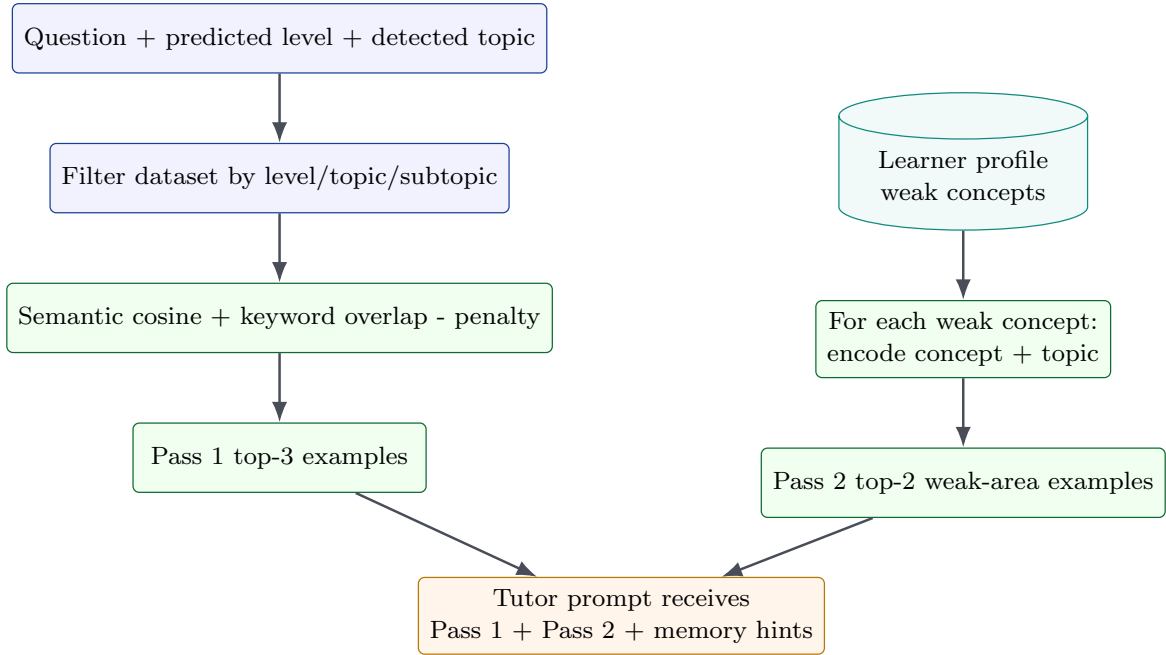


Figure 3: Two-pass RAG: query relevance plus personalized weak-area retrieval.

6.3 RAG Evaluation Results

The current repository stores a reproducible 18-question RAG evaluation checkpoint. The saved report states `RAG_TOP_N=3`. The main results are:

Layer	Metric	Saved checkpoint
Retrieval	Hit Rate@3	83.3%
Retrieval	MRR	0.8333
Retrieval	Topic Precision	83.3%
Retrieval	Level Precision	100.0%
Retrieval	Average Semantic Similarity	0.7153
Retrieval	Level Detection Accuracy in RAG sample	77.8%
Generation	Faithfulness	5.00 / 5
Generation	Answer Relevancy	5.00 / 5
Generation	Context Utilization	4.44 / 5
Generation	Level Fit	4.94 / 5

Table 8: Saved RAG evaluation checkpoint.

A later summarized evaluation run reports a stronger retrieval scorecard (Hit Rate@3 = 88.9%, MRR = 0.8889, Topic Precision = 88.9%, Level Precision = 100%, and Average Semantic Similarity = 0.7172). The conservative values in Table ?? are retained as the primary checkpoint because they are tied to the saved per-question evaluation artifacts.

6.4 Weak-Area Retrieval Results

The repository also stores `eval/pass2_eval_results.json`. It evaluates five simulated learner weakness profiles across backpropagation, RNNs, overfitting, attention, and CNNs. Results:

Metric	Value	Interpretation
Profiles evaluated	5	Five different topic/weak-concept configurations.
Retrieved examples per profile	2	Exactly two examples returned per case.
Keyword relevance	100%	Retrieved examples contained weak-area concepts.
Average semantic similarity	0.4931	Moderate similarity is expected because weak concepts are short queries.

Table 9: Pass-2 personalized weak-area retrieval evaluation.

Examples include retrieving chain-rule and gradient-flow backpropagation examples, pooling and feature-map CNN examples, and attention-score/QKV examples for advanced attention weaknesses.

7 Tutor Agent and Prompting Strategy

7.1 Tutor Prompt Assembly

The Tutor Agent uses the classifier level, detected topic, retrieved examples, weak-area examples, memory hint, evaluation strategy hint, and teaching mode. The updated code explicitly separates a system instruction from the user prompt.

7.1.1 Tutor System Message

You are EduAgent, an adaptive AI tutor specializing in {topic}.

You MUST stay focused on {topic} throughout your answer.

Do not drift into unrelated ML concepts unless {topic} itself explicitly requires them.

Pitch your explanation at {level} level.

7.1.2 Tutor User Prompt Template

QUESTION: {user_question}

TOPIC: {topic}{memory_section}{eval_section}

RETRIEVED KNOWLEDGE BASE EXAMPLES - these are your PRIMARY source:
{examples_text}{weak_rag_section}

GROUNDING RULES (follow strictly):

1. Build your answer directly upon the facts, concepts, and explanations in the retrieved examples above.
2. Every key claim in your answer should trace back to the retrieved examples.
3. You may expand or re-explain retrieved content - but do NOT ignore it.
4. Do NOT copy retrieved text verbatim - synthesize and adapt in your own words.
5. Do NOT introduce facts or concepts that contradict the retrieved examples.
6. Do NOT include citation markers or bracketed source references such as [1], [2], or [3].

LEVEL GUIDE:

- beginner: simple words, analogies, no jargon
- intermediate: moderate detail, 1-2 key terms explained

- advanced: technical depth, assume background knowledge

ANSWER FORMAT:

- Use short bullet points.
- Start with the direct answer in the first bullet.
- Use 3 to 5 bullets total.
- Keep each bullet to 1 or 2 short sentences.
- Add a final "Next step:" bullet only when a useful follow-up topic naturally fits.
- Do not use numbered citations, footnotes, or source brackets.

{mode_instruction}

Answer clearly and concisely in the bullet format above.

7.2 Adaptive Teaching Modes

7.2.1 Mode Selection Logic

Mode selection prioritizes the latest same-topic evaluation hint. If no same-topic hint exists, it falls back to profile mastery, weak areas, and topic counts.

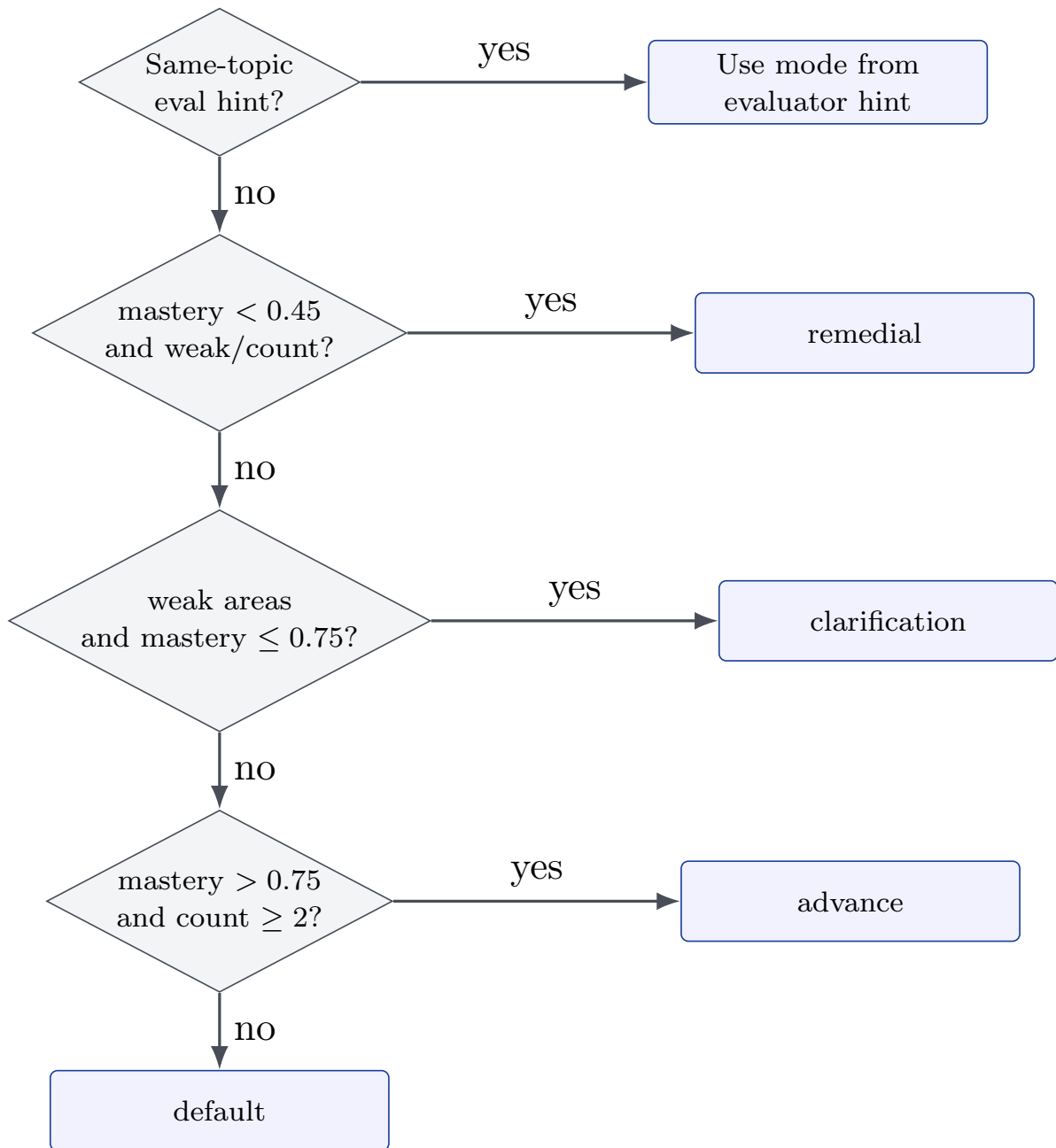


Figure 4: Teaching-mode inference in the updated Tutor Agent.

7.2.2 Mode-Specific Instruction Blocks

Remedial mode.

Mode-specific instructions:

- Re-teach from scratch.
- Use very simple wording.
- Use one small concrete example.
- Explain in 4 to 6 short sentences.
- Focus on one main idea first before adding detail.
- Avoid abstract definitions unless absolutely necessary.
- Avoid repeating the same explanation style or analogy used earlier for this topic.
- Start with: "Let's simplify it completely."

Clarification mode.

Mode-specific instructions:

- Briefly restate the main idea in one or two simple sentences.
- Then focus on the weak or confusing concept.
- Use one clear example.
- Do not repeat the whole long explanation.

- Keep the answer focused and moderately short.

Advance mode.

Mode-specific instructions:

- Assume the learner understood the basics.
- Avoid repeating the full beginner introduction.
- Give a slightly deeper explanation.
- Connect the concept to a related next-step idea.
- Keep the answer clear but more intellectually rich.

Default mode.

Mode-specific instructions:

- Give a normal level-appropriate explanation.

7.3 LLM Backend and Fallback

The updated repository uses Gemini through `google.generativeai`. `agents/llm_client.py` accepts OpenAI-style message dictionaries, maps them into Gemini roles, passes system text as `system_instruction`, and retries with exponential backoff. Rate-limit/quota errors are detected, and after repeated failures the session marks quota exhausted so later calls return fallback content immediately. Tutor fallback answers are generated from retrieved examples only; the code avoids injecting unrelated canned explanations.

8 Evaluator Agent

8.1 Follow-Up Question Prompt

The Evaluator Agent asks a short conceptual follow-up question after each tutor answer.

You are an evaluator for an adaptive AI tutor.

Topic: {topic}
Learner level: {level}

Original learner question:
{user_question}

Tutor's answer:
{tutor_answer}

Generate ONE short follow-up question to test whether the learner understood the main idea.

Rules:

- Match the learner's level.
- Focus on conceptual understanding, not memorization.
- Keep it short and clear.
- Return only the question.

If the tutor answer is a local fallback, the evaluator uses a safe hardcoded fallback: *In your own words, what is the main idea from the explanation?*

8.2 Reply Evaluation Prompt

You are an evaluator for an adaptive AI tutor.

Topic: {topic}
Learner level: {level}

Follow-up question:
{followup_question}

```

Learner reply:
{learner_reply}

Evaluate the learner's reply.

Return valid JSON only with exactly these keys:
- understanding_level: one of ["good", "partial", "poor"]
- weak_concepts: list of short concept names
- feedback: short educational feedback
- recommended_action: one of ["advance", "re-explain", "give easier example", "give more practice"]

Rules:
- Be fair and educational.
- If the learner is partly correct, mark as "partial".
- Keep weak_concepts short and specific.
- Do not include markdown.
- Return valid JSON only.

```

The output is validated with a Pydantic `EvaluationResult` schema before it can update the profile. If parsing fails, the system returns a safe `partial` result rather than incorrectly pushing the learner into remedial mode.

9 Memory Agent and Personalization

9.1 Profile Schema

The Memory Agent normalizes every profile to contain:

```

sessions, questions_asked, last_level, level_history,
topics_seen, topic_counts, weak_areas, mastery,
used_explanations, recommended_next_topics, last_evaluation

```

It also includes backward-compatibility logic for older profile shapes, such as converting an old list of weak areas into a topic-to-concept dictionary.

9.2 Mastery Update Model

The updated Memory Agent uses a named heuristic mastery model with initial score $m_0 = 0.5$:

$$\text{good: } m \leftarrow \min(1.0, m + 0.20(1 - m)) \quad (1)$$

$$\text{partial: } m \leftarrow \max(0.0, m - 0.06) \quad (2)$$

$$\text{poor: } m \leftarrow \max(0.0, m - 0.18) \quad (3)$$

The good-update rule has diminishing returns: a learner at 0.50 gains 0.10, while a learner at 0.90 gains only 0.02.

9.3 Weak-Area Logic

- **Good evaluation:** clears all weak areas for the topic if no specific concepts are flagged, or removes only the resolved concepts if the evaluator names them.
- **Partial/poor evaluation:** accumulates evaluator-provided weak concepts.
- **Repeated revisits:** if a topic is visited at least three times with no weak concept identified, the system adds `core understanding` as a weak signal.

9.4 Memory Hint Strings

The memory hint is purely deterministic profile logic, not an LLM call. It can include these lines depending on the profile:

```
The learner has seen this topic before. Do not restart with the exact same beginner introduction.
The learner has previously struggled with: {concepts}. Address these concepts carefully.
The learner appears weak in this topic. Use simpler wording, intuition, and one concrete example.
The learner appears comfortable with this topic. You may go slightly deeper than before.
Avoid repeating the exact same prior explanation style if possible. Previously used: {tags}.
No strong prior history for this topic. Start with a clear explanation.
```

9.5 Evaluation Strategy Hint Strings

Strategy hints fire only when `last_evaluation["topic"]` matches the current topic. This prevents old feedback from unrelated topics from contaminating new explanations.

```
Teaching mode: remedial. The learner did not understand the previous explanation. Do NOT repeat the same analogy-
driven explanation. Re-teach from scratch using very simple wording, one tiny concrete example, and a short
step-by-step explanation.

Teaching mode: clarification. The learner understood partially. Do not repeat the whole answer. Briefly restate
the core idea, then focus on the confusing part with one clear example.

Teaching mode: advance. The learner understood the previous explanation well. Do not repeat the full beginner
introduction. Give a slightly deeper explanation and connect it to the next related concept.
```

Recommended-action addenda can then request an easier example, re-explanation, more practice, or advancement to the next topic.

10 Application Layer, Security, and Voice

10.1 Gradio Application

The UI uses Gradio with a polished dark theme, authenticated workspace, chat panel, follow-up panel, retrieved-example display, pipeline insights, profile cards, and charts. The charts visualize mastery by topic, topic revisit count, and weak concept count by topic. The `app/main.py` callbacks orchestrate asking a question, updating the profile, generating a follow-up, evaluating the reply, and refreshing UI state.

10.2 Authentication and Storage

Passwords are not stored directly. `auth/password_utils.py` uses:

- `os.urandom(16)` for salt generation,
- PBKDF2-HMAC-SHA256 with 200,000 iterations,
- base64 encoding of `salt || hash`,
- `hmac.compare_digest` for timing-safe verification.

SQLite uses WAL mode and `synchronous=NORMAL`. Profile fields such as weak areas, mastery, topics seen, and recommendations are stored as JSON strings and loaded with safe defaults if corrupted or missing.

10.3 Voice Interaction

The updated repository includes a `voice/` module:

- `speech_to_text.py`: Whisper-based transcription wrapper using `openai-whisper`, `librosa`, and confidence derived from segment log-probabilities.
- `text_to_speech.py`: TTS wrapper that tries Piper if installed and falls back to local `pyttsx3` synthesis.
- `utils.py`: Gradio-facing glue functions for transcription and synthesis.

Voice features improve accessibility and demonstration value but are not required for the core tutoring loop.

11 Evaluation Evidence

11.1 Classifier Evaluation

The strongest quantitative result is the DistilBERT classifier: 97.92% accuracy and 97.91% macro-F1 on the balanced 240-row test split. This justifies replacing the earlier heuristic-only difficulty detection with a fine-tuned classifier plus lightweight calibration.

11.2 Dataset Evaluation

Dataset analysis shows that `eduagent_dataset_easy_v2.csv` is stronger for educational tutoring because it has 2,400 rows, 19 columns, richer metadata, more answer vocabulary, reasoning expectations, learner-confusion annotations, and no duplicate questions or answers. The 900-row baseline dataset remains useful for lightweight baseline experimentation.

11.3 RAG Evaluation

The saved evaluation checkpoint reports strong generation quality even though retrieval still has room for improvement. It shows perfect level precision in retrieved examples but topic retrieval misses in a few cases, especially where semantically adjacent topics overlap. This is an important finding: generation can be highly faithful to the provided context while topic detection/retrieval still requires refinement.

11.4 Personalization Evaluation

The saved personalization evaluation compares a clean baseline profile against a simulated experienced learner profile. It evaluates 10 questions and reports that teaching mode changed in 3/10 cases, while level detection did not change. This means personalization affects the teaching strategy rather than artificially changing the learner’s predicted level. Examples include baseline `default` mode changing to `clarification` for gradient descent, vanishing gradient, and backpropagation questions based on learner memory.

11.5 Unit Tests

The submitted repository unit-test suite reports:

```
51 passed in 1.85s
```

The tests cover authentication behavior, password hashing/verification, email validation, registration edge cases, profile normalization, backward compatibility, mastery clamping, diminishing returns, weak-area update behavior, and used-explanation deduplication.

Test file	Observed scope	Main behavior covered
tests/test_auth.py	Auth/security	Password verification, hash uniqueness, invalid email handling, duplicate-user path, and registration/login edge cases.
tests/test_memory.py	Personalization	Topic counts, mastery deltas, weak-area updates, used-explanation tags, hint generation, clamping, and diminishing-returns behavior.
tests/test_profile.py	Profile schema	Default keys, type repair, legacy weak-area conversion, profile idempotency, and JSON round-trip behavior.

Table 10: Unit-test coverage areas.

12 Comparison Against Baselines

12.1 Dataset Baseline

Criterion	Baseline dataset	Updated dataset
Rows	900	2,400
Columns	5	19
Subtopics	absent/limited	explicit
Pedagogical annotations	low	high
RAG readiness	moderate	high
Instruction tuning usefulness	moderate	stronger

Table 11: Qualitative comparison of the baseline and updated datasets.

12.2 Classifier Baseline

The classifier is compared against simple non-neural baselines: a majority-class predictor and a keyword-heuristic predictor. The fine-tuned DistilBERT model is the deployed classifier in the updated application.

Method	Accuracy	Macro-F1
Majority class baseline	33.3%	16.7%
Keyword heuristic baseline	62.1%	60.4%
Fine-tuned DistilBERT	97.92%	97.91%

Table 12: Classifier comparison against simple non-neural baselines.

12.3 Personalization Baseline

The personalization baseline evaluates whether stored learner history changes tutoring strategy without changing classifier output. The clean profile acts as the baseline; the personalized profile simulates an eight-session learner history with stored weak areas and topic counts. In the saved evaluation, 3 out of 10 teaching-mode decisions changed, while difficulty-level detection remained stable in all 10 cases.

Question	Baseline mode	Personalized mode	Level changed?	Observed personalization signal
What is gradient descent?	default	clarification	No	Prior gradient-descent history triggers focused clarification instead of a fresh default explanation.
How does a neural network learn?	default	default	No	No mode change; the memory state did not indicate a targeted weakness for this prompt.
What is the vanishing gradient problem?	default	clarification	No	Prior gradient-related weakness shifts the answer toward clarifying gradient-flow failure.
Explain the attention mechanism in transformers.	default	default	No	No mode change; topic memory did not require remediation.
What is overfitting and how do you fix it?	default	default	No	Default mode retained because profile state did not demand weak-area remediation.
How does backpropagation work?	default	clarification	No	Stored weak concepts such as chain rule and gradient flow trigger clarification mode.
What is the difference between supervised and unsupervised learning?	default	default	No	Classifier and mode remain stable for a general ML comparison.
What is a convolutional neural network used for?	default	default	No	No personalization shift was needed for this first-level CNN question.
How does the Adam optimizer improve on SGD?	default	default	No	No teaching-mode change; topic detection requires future refinement for this case.
What is transfer learning and when should you use it?	default	default	No	Default mode retained; no stored transfer-learning weakness was activated.
Summary	default-only baseline	3/10 changed	0/10	Memory changes teaching strategy in targeted cases while preserving classifier stability.

Table 13: Personalized profile vs. clean-profile baseline evaluation.

12.4 Plain LLM Tutor vs EduAgent

Capability	Plain LLM tutor	EduAgent
Difficulty classification	No explicit trained classifier	DistilBERT + calibration
Topic detection	implicit	alias + semantic cosine + fallback
Retrieval grounding	optional/manual	two-pass RAG over dataset
Learner memory	usually stateless	SQLite profile with mastery and weak areas
Follow-up evaluation	not automatic	Evaluator Agent + Pydantic validation
Adaptive mode selection	not explicit	default/remedial/clarification/advance
Progress dashboard	absent	Gradio profile + charts
Voice input/output	absent unless separately built	Whisper + TTS wrappers

Table 14: System comparison against a plain LLM question-answering tutor.

13 Deployment Considerations

For a deployed product, ordinary learners should not need to provide Gemini, Groq, OpenAI, or Ollama credentials. In the updated repository, API access is configured on the backend through environment variables such as `GEMINI_API_KEY`. Users interact only with the web UI. Ollama or local model options are best treated as developer/institutional fallback paths, not as end-user requirements.

Production note. API keys must never be hardcoded or committed. They should live in `.env` for local development and in managed secrets/environment variables for deployment. The repository already contains `.env.example` and uses `python-dotenv`/environment variables.

A production-grade version would require rate limiting, usage monitoring, admin controls, cost limits, data privacy policy, and stronger evaluation on real learner data.

14 Limitations and Threats to Validity

1. **Synthetic dataset:** The 2,400-row dataset is generated with Gemini and deterministic filters. It is structured and useful, but it is still synthetic rather than collected from real learners.
2. **Classifier generalization:** The 97.92% test accuracy is on an in-distribution split of the same generated dataset. Real student wording, spelling errors, and incomplete questions may reduce performance.
3. **RAG sample size:** The current repository RAG checkpoint has 18 questions. This is useful for project evidence but not enough for publication-level statistical confidence.
4. **LLM-as-judge bias:** Generation scores use an LLM judge. This is practical for tutoring quality but should be supplemented with human evaluation.
5. **Mastery heuristic:** Mastery updates are interpretable but not yet calibrated against real learning gains or Bayesian Knowledge Tracing.
6. **Weak-area persistence:** Weak concepts accumulate unless resolved; future versions should add decay or recency weighting.
7. **API dependency:** Tutor/evaluator quality depends on the configured LLM backend and quota availability.

15 Future Work

- Add a larger human-evaluated test set of real learner questions.
- Calibrate mastery updates with Bayesian Knowledge Tracing or Item Response Theory.
- Add weak-concept decay and time-aware learning history.
- Extend RAG evaluation from 18 examples to a 100+ question benchmark across all 25 topics.
- Add teacher/admin dashboard for classroom-level analytics.
- Support document/PDF upload as a separate RAG corpus while keeping the dataset-based RAG as the stable baseline.
- Add multimodal diagram-question input using a vision-capable model.
- Package backend API keys as deployment-side secrets so students never need to configure model providers.

16 Conclusion

EduAgent is an impressive academic prototype because it integrates several components that are often presented separately: dataset construction, fine-tuned NLP classification, semantic retrieval, adaptive prompting, learner memory, evaluation, authentication, dashboarding, and voice interaction.

The updated repository’s strongest quantitative evidence is the fine-tuned DistilBERT classifier with 97.92% accuracy and 97.91% macro-F1 on a balanced 240-row test set. The two-pass RAG evaluation shows strong grounding and level fit, while revealing remaining topic-detection/retrieval issues that are appropriate future-work targets. The Memory Agent closes the loop by converting evaluator output into mastery updates, weak-area tracking, teaching modes, and future prompt hints. Overall, EduAgent is best described as a personalized, agentic, retrieval-augmented AI tutoring system for AI/ML education, suitable as a strong B.Tech GenAI project and a foundation for further educational AI research.

A Appendix A: RAG Judge Prompt

The RAG evaluation script uses a combined LLM-as-judge prompt:

```
Rate this AI tutor response on 4 dimensions (1=worst, 5=best).

QUESTION: {question}
STUDENT LEVEL: {level}
RETRIEVED CONTEXT (what the tutor was given):
{ctx}
TUTOR ANSWER: {answer}

Scoring criteria:
- faithfulness: answer only states things supported by the context (not hallucinated)
- relevancy: answer directly and fully addresses the question
- context_use: tutor explicitly built upon or referenced the retrieved examples
- level_fit: language and depth suit a {level} student

Reply ONLY with this JSON (integers 1-5, no other text):
{"faithfulness": <1-5>, "relevancy": <1-5>, "context_use": <1-5>, "level_fit": <1-5>, "reason": "<one sentence>"}
```

B Appendix B: Core Equations

B.1 Retrieval Ranking

$$\text{final_score} = 0.75 \cdot \cos(q, d_i) + 0.25 \cdot \text{Jaccard}_{1,2}(q, d_i) - \text{penalty}(d_i)$$

B.2 Mastery Update

$$\begin{aligned} \text{good:} \quad m &\leftarrow \min(1.0, m + 0.20(1 - m)) \\ \text{partial:} \quad m &\leftarrow \max(0.0, m - 0.06) \\ \text{poor:} \quad m &\leftarrow \max(0.0, m - 0.18) \end{aligned}$$

C Appendix C: Selected Reproducibility Commands

```
# Run unit tests
python -m pytest -q tests

# Train the difficulty classifier
python train_distilbert_difficulty.py \
  --data_path datasets/data_easy/eduagent_dataset_easy_v2.csv \
  --output_dir models/distilbert_eduagent_v2 \
  --epochs 3 \
  --batch_size 8

# Run RAG evaluation
python rag_evaluation.py --n 18

# Run the application
python gradio_app.py
```

References

1. S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR, 2023.
2. V. Sanh et al. *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. NeurIPS EMC2 Workshop, 2019.
3. N. Reimers and I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP, 2019.

4. P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.
5. A. Corbett and J. Anderson. *Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge*. User Modeling and User-Adapted Interaction, 1994.
6. J. Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL, 2019.